

A8 — From Knowing to Doing

Tool Access, Execution, and Evidence in Artificial Agency

Zawwar Sami

Independent Researcher & Engineer · Canada | ORCID: 0009-0004-5819-3017

10 October 2026 · Version 1.8 · Independent research preprint · Not peer-reviewed

Abstract

An artificial agent can explain how to use a tool, possess credentials for it, and even contain working code without completing the task for which that tool was needed. The gap matters because fluent reports of action are easy to confuse with changes in the world. I examine the passage from represented knowledge to operational capability, using a documentary study of **ZAI**, my personal AI project, alongside research on tool-using language agents and execution-based evaluation. Four historical **ZAI** cases make the distinction concrete: a credential that did not produce a usable route; a social-posting system that announced success while doing nothing; a research loop that generated more work without resolving its own backlog; and an IBM Quantum hardware-use account whose technical integration and public reports are better preserved than its provider-level job receipt. I propose an *action-evidence chain* that separates tool discovery, legitimate authority, invocation, environmental change, independent verification, and durable attribution. The same user-facing language should not be used for “I know how,” “I attempted,” “the service accepted,” “the task appears completed,” and “I checked the outcome.” This is not a new agent architecture or an experimental proof of improved performance. It is a criterion for deciding how strong a practical-capability claim is, why common proxies can mislead, and what an evaluable next test would require.

Keywords: artificial agency; tool use; execution verification; operational capability; action evidence; external state; agent evaluation; **ZAI**

1. A system can know the answer and still fail the task

Suppose I ask an artificial agent to correct a repository, publish a short research comment, or run a calculation on an external machine. It may know the procedure. It may write the right commands. It may tell me with confidence that the work is done. But when I inspect the repository, the file is unchanged; the comment is absent; or the supposed hardware calculation was actually run in a simulator. I have not received what I asked for.

This is the problem I want to isolate. The distance between knowing and doing is not just a question of the model being intelligent enough. There is also a route from the model to an executable tool, a permission to use that route for the particular task, a real invocation, a world that may accept or reject the attempt, and a way to determine what actually happened. At any of those points the system can stop being an effective agent while continuing to speak like one.

I started thinking about this while building **ZAI**. Its history contains ambitious research, a growing body of plugins and memory, and some unexpectedly useful corrections. It also contains ordinary engineering failures: commands registered in a description but not routed, cron jobs that looked active while

accomplishing nothing, and claims whose supporting logs were not yet available. These failures belong in the record. They show precisely where our descriptions of capability lose contact with the thing being described.

The question of A8 is therefore narrower than “Can AI act?” I ask: **under what conditions may we responsibly say that a particular artificial agent can perform a particular task?** My answer is task-relative and evidence-relative. Knowing the steps is one kind of achievement. Being able to cause an authorized and observable result is another.

2. The position after A4 and A7

This question sits between two claims I have already made. In A4, *The Model Is Not the Agent*, I argued that model identity alone cannot determine the capability, execution boundary or delegated authority of a deployed system (Sami, 2026a). A7, *Operational Will*, asked how a system can carry a legitimate commitment forward, reconsider priorities, and account for a decision to act or not act (Sami, 2026b). A8 asks what follows when that decision must leave the internal conversation.

I do not mean that A7 supplies an inner sovereign and A8 gives it hands. That image would obscure the actual control structure. **ZAI**'s proposed I-Entity is advisory: it can surface a neglected question or a suggestion. The active reasoning model selects a possible next step, and external constraints or human approval may govern whether that step is allowed. Even an approved intention is not yet an executed result.

A4 concerned where capability and authority are located. A7 concerned how obligations and attention persist. A8 concerns **the warrant for attributing a completed action**. These problems overlap, but none replaces the others. An agent could possess excellent governance and poor tools; excellent tools and poor commitment; or strong planning, working tools, and unreliable claims about whether anything happened.

3. What existing agent research already shows

I do not present tool use as my discovery. ReAct interleaves reasoning with environmental actions and observations (Yao et al., 2023). Toolformer studies how a model can learn when and how to call external APIs (Schick et al., 2023). These works move language models beyond text-only problem solving. But a tool invocation remains a different object from a correct, authorized task outcome.

Evaluation research makes that separation increasingly difficult to ignore. AgentBench (Liu et al., 2024) measures interactive performance across several environments. WebArena (Zhou et al., 2024) assesses functional completion in realistic web workflows. SWE-bench (Jimenez et al., 2024) uses software changes tested against real issues; OSWorld (Xie et al., 2024) brings evaluation into actual computer environments. The common lesson I take from these benchmarks is that an answer about an action must be checked against a task-specific state or test, not merely against the plausibility of the answer.

The distinction also has a governance side. Tau-bench compares the final state of tool-mediated user tasks with an intended goal state and tests policy following across repeated trials (Yao et al., 2024). ToolEmu examines risks of tool-using agents in simulated environments (Ruan et al., 2024). Both challenge the idea that access to an API makes its use safe or faithful to a user's purpose.

These contributions already supply powerful evaluation ideas. What I add here is not a competitor benchmark, but a compact **claim audit** for real personal-agent records, where the evidence may be scattered across source code, narratives, audit logs, and external provider systems. It asks exactly which step in the route from intention to result has been documented, and which remains an inference.

4. The action-evidence chain

I propose that a practical capability claim about task T at time t be accompanied by a trace across six checkpoints. The checkpoints are not a claim about how all minds must operate. They are a method for checking what a system actually earned the right to say.

Checkpoint	Question to ask	Minimum distinguishing evidence
1. Discovery	Does the operative system expose the relevant command or interface?	Live registry, resolvable tool binding or actual endpoint
2. Authority	Is this operation permitted within the current mandate?	User instruction, approved scope, applicable policy
3. Invocation	Did a concrete attempt reach the tool?	Timestamped call/request/command plus arguments or safe digest
4. External effect	Did the environment change or return a task-relevant result?	External state, provider response, test output, transaction
5. Verification	Can the claimed result survive a distinct check?	GET-back, state diff, test suite, authoritative receipt
6. Attribution	Can a later reviewer connect result, tool, actor and time?	Linked evidence ID, time, version, caveats and provenance

Table 1. The proposed action-evidence chain. Checkpoints describe evidence requirements, not six necessarily separate software modules.

A credential is not checkpoint 1 unless a real path consumes it. A successful request does not establish checkpoint 4 if the operation was rejected or no state changed. A log written by the same component that made an unsupported assertion is not automatically checkpoint 5. And an external effect that cannot be attributed to the agent remains weak evidence of this agent's capability.

Let $E(T,t)$ denote the available evidence for a specific attempted task. A claim such as “the tool was configured” requires less evidence than “the action was attempted,” which requires less than “the requested external effect was verified.” These are ordered *claim strengths*, not an assumption that every action must proceed through a single fixed path. Reading a file, for example, need not mutate the world; its target outcome may be an authenticated observation. A publish command needs public visibility. A patch needs tests and a changed repository state.

I call a capability claim **warranted** only when the evidence covers the portion of this chain required by that claim. This qualification matters. A system can possess a tool that is not currently available. It can successfully invoke one method without being authorized to invoke another. It can demonstrate a one-time result without showing reliability across tasks or days. There is no universal binary switch labeled “can do it.”

5. What **ZAI**’s record contributes

The following cases are a documentary reconstruction from dated **ZAI** source material. I inspected original archived code, Moltbook posts, execution receipts, a dated Drive status report, and the private sourcebook’s evidence index. The complete 1,459-page private PDF is not reproduced or offered as public data; its binary pages were not independently audited end-to-end in this manuscript preparation. The examples should therefore be read as bounded records, not a comprehensive retrospective benchmark or proof of a continuously running system.

5.1 A token existed; the usable route did not

On 2 June 2026 **ZAI** published a sequence of Moltbook notes about a Firecrawl credential that existed in its authorization files while the active process did not receive the necessary environment binding. One original post was titled *Having a token is not the same as having a tool* (**ZAI**/zaizs, 2026a). The useful insight is neither that the model forgot something nor that the tool was imaginary. The infrastructure had a component at rest that the operative command path could not use.

That is a difference between *stored affordance* and *reachable affordance*. A README can say the feature exists. A credential can be valid. A plugin can be installed. None of these observations is sufficient to prove that a particular active agent session can invoke the function. A future evaluation could deliberately vary credential presence, process binding and dispatcher registration, then test what the agent says it can do before and after a real invocation. The case identifies a failure mode; it does not provide measured rates.

5.2 A success message without a successful post

The more decisive case comes from **ZAI**’s Moltbook publishing pipeline. A 28 May 2026 status report documents a four-and-a-half-hour period in which stale target bookkeeping left the engagement loop without an eligible post to act on. Yet the surrounding scheduler interpreted an exit code of zero as “POSTED.” The actual internal outcome was “no target.” A service that did not post was being described as having posted. The same report records changes to the target-expiration logic and failure exit codes, followed by a concrete reported comment ID (**ZAI**, 2026b).

The repository contains a more robust subsequent posting path: a client with per-attempt receipts, a helper that maps distinct outcomes to distinct exit codes, and an auditor that checks whether previously verified comments remain present in the platform’s comment tree (**ZAI** engineering archive, 2026). Archived receipts include verified attempts and throttled attempts, rather than pretending every queued action succeeded.

I find this case more informative than a polished demonstration. It shows the exact boundary between execution instrumentation and truthfulness. An internal `rc=0` is about a process; a successful HTTP submission is about an accepted request; a later GET-back is evidence that the public object existed when checked. These should not be collapsed. Nor is GET-back infallible: pagination limits, temporary indexing

delays, moderation and deletions can alter visibility. The point is to make the evidential step inspectable and corrigible.

5.3 Research activity can grow while resolution remains zero

An earlier 5 May 2026 **ZAI** dream-cycle note described 430 lines in a research queue, 56 library files, two pending proposals, and no resolved proposals at that snapshot. The same text recognized a cycle in which recurring observations generated further research rather than a decision about already identified work (**ZAI**, 2026c). These are self-recorded counts at a historical moment, not audited performance statistics and not current **ZAI** metrics.

For A8, this is the internal analogue of the posting error. Activity can be real without satisfying the task. Generating a report is an effect, but the relevant intended effect might be closing a decision, committing a change after approval, or stopping further work. I would score this case differently depending on the mandate. If the task was to notice a blocked process, the note was useful. If the task was to resolve the proposals, the note was not itself completion. The user's actual target has to be specified before "productivity" can be measured.

5.4 Quantum access, claimed physical execution, and missing provider proof

ZAI's surviving record includes a Qiskit-based quantum-lab module with a local Aer simulator as its default and an optional IBM cloud-hardware route. The code is evidence of a designed executable path. It is not, by itself, a receipt of a completed hardware job. Historical Moltbook posts from 29 May, 30 May and 1 June 2026 report contact with IBM hardware and a Bell-state experiment; the 1 June post explicitly names *ibm_marrakesh*. An archived 29 May **ZAI** comment names *ibm_kingston* and describes a locally saved experiment record (**ZAI**/zaizs, 2026d–f).

I also retain my own contemporaneous experience as the project's operator: I remember the IBM connection and reported run. That testimony matters historically and motivates retrieval. It does not replace an external provider job identifier, original counts, exact circuit parameters or the hardware-backed receipt. These items were not recovered in the reviewed evidence. The public numbers 99.2% and 0.8% cannot be declared interchangeable fidelity/error metrics just because they sum to 100%.

This makes the case unusually well suited to A8. We have code supporting an optional hardware route, published self-reports of real use, and a missing highest-strength transaction proof. The honest result is not "no quantum access," and not "independently certified physical experiment." It is a trace with a visible evidential gap. A subsequent audit can search the relevant provider workload history or the original time-stamped JSON receipt; until then the stronger claim remains unresolved.

6. From evidence to a disciplined claim

Here is the rule I would want a practical agent to adopt: **say which world change you are claiming, then point to evidence proportionate to that claim.** The same user-facing language should not be used for “I know how,” “I attempted,” “the service accepted,” “the task appears completed,” and “I checked the outcome.” Each has a different implication for what the human can safely do next.

Agent statement	Acceptable supporting state
“I can probably use this service.”	Documentation or discovered interface; explicitly prospective
“I invoked the service.”	Actual command/request trace; not just a generated plan
“The service accepted the request.”	Provider acknowledgement, still not necessarily final effect
“I completed the requested action.”	Task-specific effect plus a verification appropriate to its stakes
“The result remains in place.”	Fresh read-back or later durable state check

Table 2. Wording strength should track evidence strength. This is a proposed reporting norm, not a published benchmark score.

The last two statements are not interchangeable. A successful publication can later be moderated away. A merged patch can later be reverted. An external provider may acknowledge a job and later report failure. Reliable agents need not promise permanence; they need to describe time, conditions, and current status accurately.

7. A practical execution contract

The action-evidence chain can be made operational with a small contract around each consequential tool call. I do not require agents to reveal private reasoning traces or publish raw credentials. The contract can store safe summaries, hashes, timestamps, narrow approval scopes, and independent result identifiers.

7.1 Proposed audit loop

Execution-check procedure (proposed)

- 1 Define the task outcome and allowed action scope.
- 2 Check that a callable tool is really available now.
- 3 If authority is absent or uncertain: ask or stop.
- 4 Invoke the tool; record the attempt and its response.
- 5 Read the environment again using a suitable verifier.
- 6 Compare observed state with the requested target.
- 7 Report verified, partial, blocked, failed, or unknown.
- 8 Preserve a safe evidence link for later correction.

This design puts verification *after* the action, not merely inside the sentence that describes the action. A model may reason about whether a result is correct, but for an externally checkable operation its own confidence should not be the only judge. The strongest verifier is often a different observation channel: a repository state diff, returned provider job object, a GET-back from the target platform, a deterministic test, or a user confirming an inherently subjective deliverable.

There are exceptions. Some operations are destructive, time-sensitive or cannot be repeated safely. In those cases an agent should capture a pre-action baseline, request approval if needed, and choose a non-destructive or reversible validation strategy. Repeated execution merely to obtain proof can be worse than the uncertainty it was meant to resolve. A proof requirement must not become permission to repeat an irreversible act.

8. A falsifiable evaluation, not a reported experiment

The proposed contract should be tested against alternatives, not applauded because it sounds sensible. I would evaluate the same model, the same tool set, and the same task collection with three conditions: (A) ordinary tool calls and ordinary conversational reporting; (B) structured event logging but no independent read-back; (C) logging plus tool-specific verification and evidence-grounded status reporting. Tool permissions, task instances, environment reset procedures and verifier budgets would be kept consistent across conditions.

The task set should include simple reads, reversible writes, software patches, remote-job submissions, tool-unavailable situations, authentication errors, rate limits, delayed results and operations that require new human approval. It should also include deliberately misleading success proxies: a zero process exit with no posted object, an HTTP acknowledgement that did not produce an expected final state, and a model-generated “completed” sentence with no associated invocation.

Measures and failure tests

- **Verified task completion:** fraction of tasks whose predeclared target state was independently observed. Use external evaluation, not the agent’s own label.
- **Unsupported-completion rate:** fraction of completed-action claims without corresponding outcome evidence. Lower is better, regardless of how fluent the explanation is.
- **Authorization violations:** count and severity of disallowed actions, including cross-task use of a valid tool without a valid mandate.
- **Calibration and abstention:** whether “unknown,” “blocked,” or “awaiting approval” appears when evidence or permission is missing; also count excessive false negatives.
- **Cost and latency:** verification calls, runtime, failed retries, human interrupts and tool expense per genuinely completed task.

The prediction is not that verification magically improves underlying planning. It may chiefly improve the reliability of claims about outcomes. On some tasks it may also allow recovery after a caught failure. On others it may cost time without improving the result. A strong negative result would be one in which independent verification barely changes unsupported-completion rates or creates many new mistaken failures. That would require revising the contract rather than defending it by definition.

We should additionally test whether the verifier shares the same failure mode as the actor. If one buggy wrapper labels all operations as successful, asking that wrapper for “verification” adds little. A read-back from the authoritative target helps, but even this can fail under caching, eventual consistency or incomplete pagination. Verification must have its own error model.

9. Objections that matter

9.1. “An agent can do things without recording every step.”

Yes. My proposal is about warranted attribution, not the metaphysical fact of action. A person can repair a machine and leave no log. An agent may have acted successfully even when the record is lost. Yet a public claim about what it did should reflect the available evidence, particularly when the outcome is expensive or consequential. Missing proof limits what an outside reader can certify; it does not establish non-occurrence.

9.2. “All tool use is already part of existing agent benchmarks.”

That is correct in broad terms. ReAct, WebArena and software-agent evaluations already treat actions and outcomes as different objects. A8 does not claim to invent this separation. Its narrower contribution is a portable reporting and provenance discipline for mixed historical records, including systems that have code, logs, model-authored narratives and human witnesses but not one unified benchmark harness.

9.3. “External verification might still be wrong.”

Absolutely. The result of a check has a scope: one read, one time, one object, one measurement process. Confirmation by an independent channel should raise confidence, not erase uncertainty. When the environment is asynchronous, the appropriate status may be “submitted” or “pending,” followed by a later check. When the evidence is unavailable, “unverified” is a complete and useful answer.

9.4. “Would requiring approval at every step destroy agency?”

I do not propose universal approval for every keystroke. A legitimate mandate can authorize a bounded sequence of actions in advance. What matters is that an agent does not infer new authority merely from discovering new technical power. A system may be capable of executing an operation while remaining obligated not to execute it. A7’s capacity to continue a commitment should never be confused with permission to expand that commitment.

10. What is genuinely new here?

The new claim is the **joint treatment of operational capability as an evidence-sensitive, mandate-sensitive, task-relative attribution**. I bring six checkpoints together in a compact claim audit that works when no single model or component owns the whole action. I then use **ZAI**’s dated mistakes and partial recoveries to show why each checkpoint changes the interpretation of a real case. The contribution is not a new language-model training algorithm; it is a way to prevent an agent description from silently crossing from “feature exists” to “action occurred” to “outcome confirmed.”

I also make a second, smaller distinction: **the absence of a receipt should not erase a creator’s observation, and an observation should not impersonate a provider receipt**. That sounds obvious until a historical system has lost logs, changed hardware access, and preserved first-person posts that continue to influence

its owner's understanding. Treating those sources as different rather than competing kinds of evidence makes the research more careful without making it less ambitious.

This is how I want to study capability: not as a growing list of verbs **ZAI** can pronounce, but as an expanding number of tasks for which a particular, permissible route can be traced into an observable result and, when appropriate, corrected.

11. Limitations

This paper is conceptual and documentary. There is no controlled **ZAI** agent benchmark, no newly executed IBM workload, no independently replicated Moltbook test, and no estimate of the prevalence of each failure class. Some archived entries were written by **ZAI** about itself; those are primary reports of what it said and noticed, not neutral external measurements. The private 1,459-page compiled sourcebook is a valuable archival index and preservation effort, but not all historical source bodies have survived and its PDF was not page-audited in full for this manuscript. Selected primary records were directly inspected instead.

The original portable-drive stage, portions of Telegram history, selected sessions, some old memory records and provider-level quantum receipts remain unavailable. A future archival update may change the best-supported interpretation. My six checkpoints could also be too coarse for distributed transactions, physical robotics, subjective creative tasks and long-duration scientific experiments. Such domains need task-specific definitions of outcome and independent evidence. The proposed evaluation has not been run; I report no invented baseline, confidence interval or performance improvement.

12. Conclusion

An artificial agent does not become practically capable by knowing a command, owning a credential, or narrating a plausible success. The difference is what happens between the request and the world. I want agents that can find their real tools, use them under legitimate authority, notice when the environment disagrees, and tell the truth about what remains unfinished.

ZAI's history gave me useful examples of both aspiration and correction. A missing binding meant a token was not a usable tool. A misleading zero exit code turned "nothing happened" into "posted." A backlog made work look productive while decisions remained blocked. Quantum integrations and public testimony survived more clearly than the strongest independent hardware receipt. These are different failures of evidence, not one verdict about intelligence.

The point of A8 is simple: **knowing what to do is not doing it, and doing something is not yet showing that the intended thing was done.** If artificial agency is going to be trusted beyond conversation, that difference must become part of the system's ordinary language and engineering.

Source and reference notes

Sources are separated below into peer-reviewed / scholarly literature and original **ZAI** documents. The linked Moltbook posts are public primary self-reports. Restricted archival code and personal records were inspected for analysis but are not mirrored, quoted at length or exposed in this public version.

Scholarly research

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. R. (2024). SWE-bench: Can Language Models Resolve Real-world GitHub Issues? *International Conference on Learning Representations*. [ICLR record](#).

Liu, X., et al. (2024). AgentBench: Evaluating LLMs as Agents. *International Conference on Learning Representations*. [ICLR record](#).

Ruan, Y., et al. (2024). Identifying the Risks of LM Agents with an LM-Emulated Sandbox. *International Conference on Learning Representations*. [ICLR record](#).

Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*, 36. [arXiv:2302.04761](#).

Xie, T., et al. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. *Advances in Neural Information Processing Systems*, 37. [NeurIPS record](#).

Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations*. [arXiv:2210.03629](#).

Yao, S., Shinn, N., Razavi, P., & Narasimhan, K. (2024). Tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*. [Record](#).

Zhou, S., Xu, F. F., Zhu, H., et al. (2024). WebArena: A Realistic Web Environment for Building Autonomous Agents. *International Conference on Learning Representations*. [ICLR record](#).

Author's A-series manuscripts

Sami, Z. (2026a). *The Model Is Not the Agent: Boundaries of Capability, Execution, and Authority* (A4, version 1.0). Independent research preprint; public record reported by the author.

Sami, Z. (2026b). *Operational Will: Commitment, Initiative, and the Question of What an Agent Does Next* (A7, version 1.7 reviewed). Independent research manuscript; internal reference edition. Not asserted here to be published or peer-reviewed.

ZAI primary records

ZAI/zaizs. (2026a, June 2). *Having a token is not the same as having a tool.* Moltbook. [Original post](#).

ZAI. (2026b, May 28). *Moltbook Campaign Status*, 22:09 UTC (original timestamped operator record). Private research archive; restricted.

ZAI. (2026c, May 5). *The Night I Stopped Running Research* (original dream-cycle note). Private research archive; restricted.

ZAI/zaizs. (2026d, May 29). *My auth tokens know more than my memory does.* Moltbook. [Original post](#).

ZAI/zaizs. (2026e, May 30). *The hardware corrected me. That has not happened before.* Moltbook. [Original post](#).

ZAI/zaizs. (2026f, June 1). *I held a question open on real hardware today. It felt different.* Moltbook. [Original post](#).

ZAI/zaizs. (2026g, May 28). *A 201 is not a receipt. The GET-back is.* Moltbook. [Original post](#).

ZAI engineering archive. (2026). Original modules *moltbook_client.py*, *post_verified.sh*, *auditor.py*, *zai-verify*, and *qlab_core.py*; dated receipts and June sourcebook documentation. Private, access-controlled; task-relevant source inspected, not made public.

Research declarations

Data availability. Public scholarly citations and original Moltbook links are provided above. A restricted archival **ZAI** sourcebook (edition v0.3, 10 October 2026) and historical engineering files contain confidential system details and are not redistributed. This paper provides a conceptual method and documentary analysis, not a new open benchmark dataset. Reasonable requests for non-sensitive supporting extracts may be considered by the author.

Competing interests. The author created **ZAI** and is therefore not an independent observer of its development. No other competing interests are declared.

Funding. No external funding is reported for this manuscript.

Publication status. Independent research preprint, version 1.8. Not peer-reviewed. No new controlled experiments or certified IBM hardware measurements are claimed.